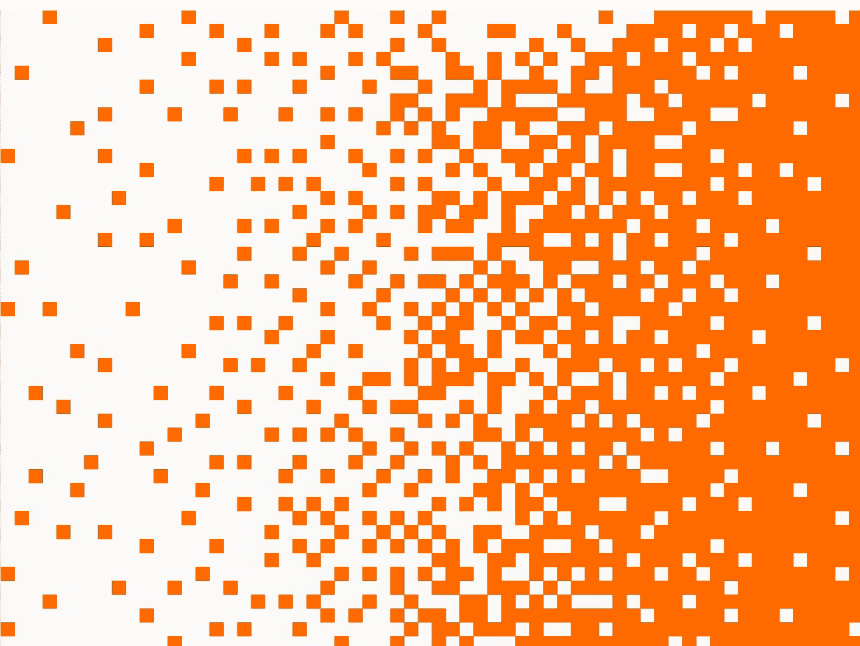


OpenAI API und lokales LLM mit LM Studio · Grundlagen, Architektur,
Fehlerprüfung und Betrieb

RAG für eine Waschmaschinenanleitung



Eine Anleitung, eine App, zwei Suchwege

Eine Anleitung, eine App und zwei Suchwege bilden die Fallstudie



Die Wissensbasis ist eine Siemens-Gebrauchs- und Aufstellanleitung mit 48 PDF-Seiten. Eine Flask-App auf Port 3001 beantwortet Fragen dazu und wechselt den Modellanbieter per Toggle zwischen LM Studio und OpenAI API. Hybrid-Retrieval ist der Standardweg, PageIndex die experimentelle Alternative. Lernziel ist, jede Antwort bis zum verwendeten Handbuchabschnitt zurückzuverfolgen.

Wissensbasis

➡ Eine Siemens-Gebrauchs- und Aufstellanleitung mit 48 PDF-Seiten, als Markdown extrahiert.

Ein Projekt

➡ Eine Flask-App auf Port 3001; der Modellanbieter wird im GUI per Toggle gewählt.

Zwei Suchwege

➡ Hybrid-Retrieval als Standard, PageIndex als experimentelle Alternative.

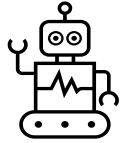
Lernziel

➡ Jede Antwort bis zum verwendeten Handbuchabschnitt nachvollziehen.

Quelle: Eigene Darstellung nach README.md, github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

RAG sucht, bevor das Modell antwortet

Ein Sprachmodell erzeugt Text aus Trainingswissen und Kontext



Ein Large Language Model berechnet Token für Token die wahrscheinlichste Fortsetzung eines Textes. Was es dabei weiß, stammt aus zwei Quellen: aus den Gewichten, die beim Training entstanden sind, und aus dem Kontextfenster, also dem Text, der ihm mit der Anfrage übergeben wird. Steht die Bedeutung von E:18 in keiner der beiden Quellen, erzeugt das Modell trotzdem eine flüssige Antwort — das ist die Halluzination. RAG füllt das Kontextfenster mit Handbuchtext.

Nur Trainingswissen

Frage: "Was bedeutet E:18 an meiner Waschmaschine?"

Das Modell kennt Siemens-Fehlercodes höchstens aus zufälligen Trainingstexten. Es antwortet dennoch — plausibel formuliert, aber unbelegt.

Trainingswissen plus Handbuchkontext

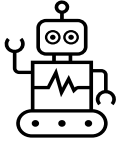
Kontext: "Anzeige E:18 — Laugenpumpe verstopft. Laugenpumpe reinigen. Seite 30"

Das Modell formuliert die Antwort aus dem mitgelieferten Text; die Quelle ist benennbar und prüfbar.

Kontextfenster: Begrenzter Platz je Anfrage — bei gpt-4.1-mini 1.047.576 Tokens, bei lokalen Modellen meist deutlich weniger. Deshalb kommt nicht das ganze Handbuch hinein, sondern eine Auswahl.

Quelle: Lewis et al. (2020), arXiv:2005.11401 (parametrisches und nicht-parametrisches Gedächtnis); developers.openai.com, gpt-4.1-mini

RAG sucht zuerst im Dokument und lässt das Modell dann antworten



Retrieval-Augmented Generation verbindet ein generatives Sprachmodell mit einer externen Wissensquelle. Die passenden Textstellen werden zur Anfragezeit gesucht und dem Modell als Kontext mitgegeben; die Gewichte des Antwortmodells bleiben unverändert. Die Wissensquelle lässt sich deshalb aktualisieren, ohne das Modell neu zu trainieren. Eine Garantie gegen Halluzinationen ist RAG nicht: Das Modell kann den Kontext falsch wiedergeben.



Zur Laufzeit: Suche und Antwort geschehen bei jeder Anfrage; das Modell selbst wird nicht verändert, nur sein Kontext.

Quelle: Lewis et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020, arXiv:2005.11401

RAG liegt zwischen Volltext-Prompt und Fine-Tuning



Der ganze Text im Prompt braucht ein Kontextfenster von Dokumentgröße und kostet bei jeder Anfrage Eingabetokens. Fine-Tuning passt die Modellgewichte an und braucht Trainingsdaten und einen Trainingslauf. RAG legt nur ausgewählte Textstellen in den Kontext; der Aufwand liegt in Aufbereitung, Index und Suche. Die Fallstudie aktualisiert fachliches Wissen über die Anleitung und ihre Indizes, nicht über das Modell.

Ganzer Text im Prompt

Wissen zur Anfrage: Dokument vollständig im Kontext

Aufwand: Kontextgröße und Eingabetokens je Anfrage

RAG — hier eingesetzt

Wissen zur Anfrage: Ausgewählte Textstellen im Kontext

Aufwand: Aufbereitung, Index und Suche

Fine-Tuning

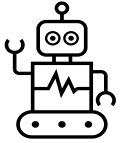
Wissen zur Anfrage: Anpassung der Modellgewichte

Aufwand: Trainingsdaten und Trainingslauf

Folge: Ein aktualisiertes Handbuch wirkt bei RAG sofort nach dem Neuaufbau des Index; Fine-Tuning müsste dafür neu trainiert werden.

Quelle: Lewis et al. (2020), arXiv:2005.11401; Gao et al. (2023/2024): Retrieval-Augmented Generation for LLMs — A Survey, arXiv:2312.10997

Der Prompt trennt Regeln, Frage und Handbuchauszüge



Das Modell erhält zwei Nachrichten. Die Systemrolle setzt die Regeln: nur belegte Antworten, Sicherheits- und Kundendiensthinweise erhalten, ein festes Antwortformat. Die Nutzerrolle trägt Frage und Handbuchauszüge als JSON-Daten, damit Anweisung und Material nicht vermisch werden. Ein Zeichenbudget von 14.000 lehnt übergroßen Kontext ab. Die Antwort kommt in drei Tags zurück; die App prüft sie und wandelt Schritte in Checkboxes.

System-Rolle — die Regeln (sinngemäß)

Nur aus den Handbuchauszügen antworten.
Sicherheits- und Kundendiensthinweise
wörtlich erhalten. Antwort in den Tags
<summary> <manual_intro> <manual_steps>.
Fehlt die Information: das sagen.

User-Rolle — Frage und Material als JSON

```
{ "frage": "Was bedeutet E:18?",  
  "auszuege": [  
    { "abschnitt": "Hinweise im Anzeigefeld",  
      "seite": 33, "text": "Anzeige: E:18; ..." },  
    ... höchstens 5 Auszüge, ≤ 14.000 Zeichen ] }
```

<summary>

➔ Kurzantwort in einem Satz — wird im GUI zuerst angezeigt.

<manual_intro> <manual_steps>

➔ Einleitung und Schritte aus dem Handbuch; die App wandelt Schritte in Checkboxes. Am Tokenlimit abgeschnitten = Fehler.

Quelle: server.py (_messages, _payload, CONTEXT_MAX_CHARS) im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Aufbereitung läuft je Handbuchstand, die Suche bei jeder Frage



Die Aufbereitung läuft nur, wenn sich die Anleitung ändert: Docling wandelt das PDF in Markdown, ein Splitter zerlegt es in Textabschnitte, das Embedding-Modell erzeugt Vektoren und LlamaIndex speichert den Index. Bei jeder Frage laufen Suche, Reranking, Kontextprüfung und die Antwort des LLM. Der erste Download der Modelle braucht Internet; danach arbeitet die Aufbereitung mit den vorhandenen Gewichten.

Aufbereitung — bei geänderter Anleitung



Bearbeitung — bei jeder Frage



Beide Phasen teilen sich das Embedding-Modell: Der Fragevektor entsteht mit demselben E5-Modell wie die Textvektoren des Index.

Quelle: Eigene Darstellung nach github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM und docling-project.github.io/docling (Abruf 19.09.2026)

Vom PDF zum durchsuchbaren Index

Ein lokaler Dateispeicher genügt für die eine Anleitung



Das Repository hält vier Bestände: das 48-seitige Original-PDF als Referenz, das extrahierte Markdown als Quelle des Retrievals, den Ordner storage/ mit Vektoren, Nodes und Index-Metadaten sowie pageindex_tree.json mit 187 Abschnitten und Quellhash. Der SimpleVectorStore von LlamaIndex lädt die JSON-Dateien beim Start in den Prozess. Eine Chroma-, Qdrant-, PostgreSQL- oder Neo4j-Anbindung gibt es im geprüften Code nicht.

Datei / Ordner	Inhalt	Verwendung
siemens-handbuch.pdf	48-seitige Originalanleitung	Referenz für jede Prüfung
siemens_wissen.md	Strukturiert extrahierter Text	Quelle des Retrievals
storage/	default__vector_store.json, docstore.json, index_store.json	LlamaIndex-Suche
pageindex_tree.json	187 Abschnitte, Summaries und Quellhash	LLM-Auswahl von Abschnitten

Quelle: github.com/swrobut/SiemensWashingMachineTroubleShooting_LocalLLM; developers.llamaindex.ai, Persisting & Loading Data (Abruf 19.09.2026)

Docling liefert Markdown mit Seitenmarkern, das geprüft werden muss



Docling, ein Projekt aus IBM Research, erkennt Layout, Lesereihenfolge und Tabellenstruktur eines PDF und exportiert sie als Markdown. Der überarbeitete Parser verarbeitet jede Seite einzeln und schreibt den Marker `<!-- pdf-page: N -->` davor, damit die Fundseite belegbar ist. Eine neue Extraktion wird erst gegen das PDF geprüft, bevor sie die aktive Wissensdatei ersetzt. Bis dahin bleibt das bisherige Markdown die Wissensbasis.



Prüfpunkte vor dem Ersetzen der Wissensdatei

Lesereihenfolge der Spalten und Kästen · **Tabellenzeilen** vollständig und richtig zugeordnet · **Symbole** und Sonderzeichen · **Warnhinweise** ohne Verlust · **Seitenherkunft** über die Marker belegbar. Ein geänderter Inhalt invalidiert Vektorcache und PageIndex-Baum.

Quelle: docling-project.github.io/docling/; `parser.py` im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Ein Token ist ein Wortstück, kein Wort



Transformer-Modelle lesen keinen Text, sondern Nummern aus einem festen Vokabular. Der Tokenizer zerlegt Text dafür in Subwörter: Häufige Wörter bleiben ein Token, seltene zerfallen in Stücke. E5 nutzt das SentencePiece-Vokabular von XLM-RoBERTa mit rund 250.000 Einträgen. Deshalb zählen Tokenlimits wie 512 nicht Wörter — ein deutsches Kompositum kostet mehrere Tokens, und 440 Tokens sind weniger Text, als die Zahl vermuten lässt.

"Laugenpumpe"

4 Tokens

_Lau	gen	pump	e
------	-----	------	---

"Laugenpumpe verstopft."

8 Tokens

_Lau	gen	pump	e	_ver	stop	ft	.
------	-----	------	---	------	------	----	---

"query: Was bedeutet der Fehlercode E:18?"

11 Tokens

_que	ry	:	_Was	_bedeutet	_der	_Fehler	code	_E	:18	?
------	----	---	------	-----------	------	---------	------	----	-----	---

Warum das zählt

440 Tokens je Chunk entsprechen bei diesem Vokabular grob 200 bis 300 deutsche Wörter. Die Fehlercodetabelle passt so nicht als Ganzes in einen Chunk.

Modellabhängig

Jedes Modell hat sein eigenes Vokabular. Der Splitter zählt deshalb mit dem E5-Tokenizer, nicht mit dem des Antwortmodells oder nach Zeichen.

Quelle: Tokenisierung mit tokenizer.json von intfloat/multilingual-e5-small (Tokenizers 0.x, 19.09.2026); huggingface.co/docs/transformers/tokenizer_summary

Tabellen werden zeilenweise zerlegt und behalten ihre Kopfzeile



Die Störungs- und Fehlercodetabellen des Handbuchs sind breit; als Ganzes eingebettet überschreiten sie das Tokenlimit, und ein einzelner Fehlercode geht in der Masse unter. Der Splitter zerlegt große Tabellen deshalb zeilenweise: Jede Datenzeile bekommt die Abschnittsüberschrift und die Spaltennamen mit, Fortsetzungszeilen übernehmen die Bezeichnung der vorigen Zeile. Mehrere Tabellen behalten jeweils ihre eigenen Spaltenüberschriften.

Anzeige	Ursache / Abhilfe
E:18	Laugenpumpe oder Ablauf verstopft
E:23	Wasser in der Bodenwanne / Undichtigkeit

Eine Zeile, ein Chunk: Aus der Tabellenzeile E:18 wird ein eigener Abschnitt mit Überschrift und Spaltennamen — so findet der exakte Lookup den Code, und der Vektor beschreibt nur diese eine Störung.



Quelle: siemens-handbuch.pdf, Seite 33, Tabelle "Hinweise im Anzeigefeld" (für die Darstellung verkürzt); rag_engine.py

440 Tokens je Chunk halten Abstand zum Limit von 512



Das Embedding-Modell multilingual-e5-small verarbeitet höchstens 512 Tokens; längerer Text wird abgeschnitten und fehlt im Vektor. Zu kleine Chunks verlieren dagegen Überschrift, Bedingung oder Warnhinweis, die zur Aussage gehören. Der Splitter arbeitet mit dem E5-Tokenizer, 440 Tokens je Chunk und 40 Tokens Überlappung, damit ein Satz an der Schnittkante in beiden Nachbarn vollständig vorkommt. Die Fundseite bleibt als Metadatum am Chunk.

Zu groß

Das Modell schneidet nach 512 Tokens ab. Details am Ende des Abschnitts fehlen im Vektor.

Zu klein

Die Aussage verliert Überschrift, Bedingung oder Warnhinweis, die zu ihr gehören.

Gewählt: 440 / 40

440 Tokens je Chunk, 40 Tokens Überlappung, gezählt mit dem E5-Tokenizer.

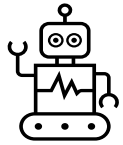
Überlappung von 40 Tokens: Der Abschnitt wird in Fenster geschnitten, die sich am Rand überdecken



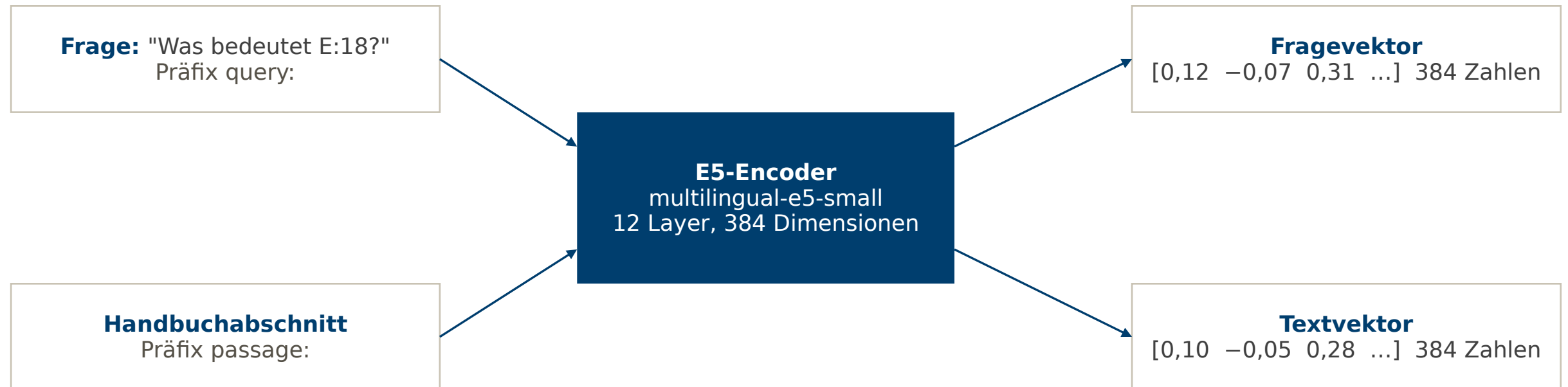
Die grau hinterlegten Ränder von 40 Tokens stehen in beiden Nachbar-Chunks. Ein Satz an der Schnittkante geht so nicht verloren, kostet aber Speicher und doppelte Treffer.

Quelle: huggingface.co/intfloat/multilingual-e5-small (Modellkarte); `rag_engine.py` im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Frage und Abschnitt werden mit demselben Modell zu 384 Zahlen



Ein Embedding-Modell bildet Text auf einen Zahlenvektor ab; multilingual-e5-small erzeugt 384 Dimensionen. Frage und Handbuchabschnitt müssen durch dasselbe Modell laufen, sonst liegen ihre Vektoren in verschiedenen Räumen und der Vergleich ist bedeutungslos. E5 verlangt Präfixe: Fragen tragen query:, Textpassagen passage: — laut Modellkarte auch bei nicht-englischem Text. Das Fehlen der Präfixe war im Entwurf vom Juli 2026 der größte einzelne Retrieval-Verlust.



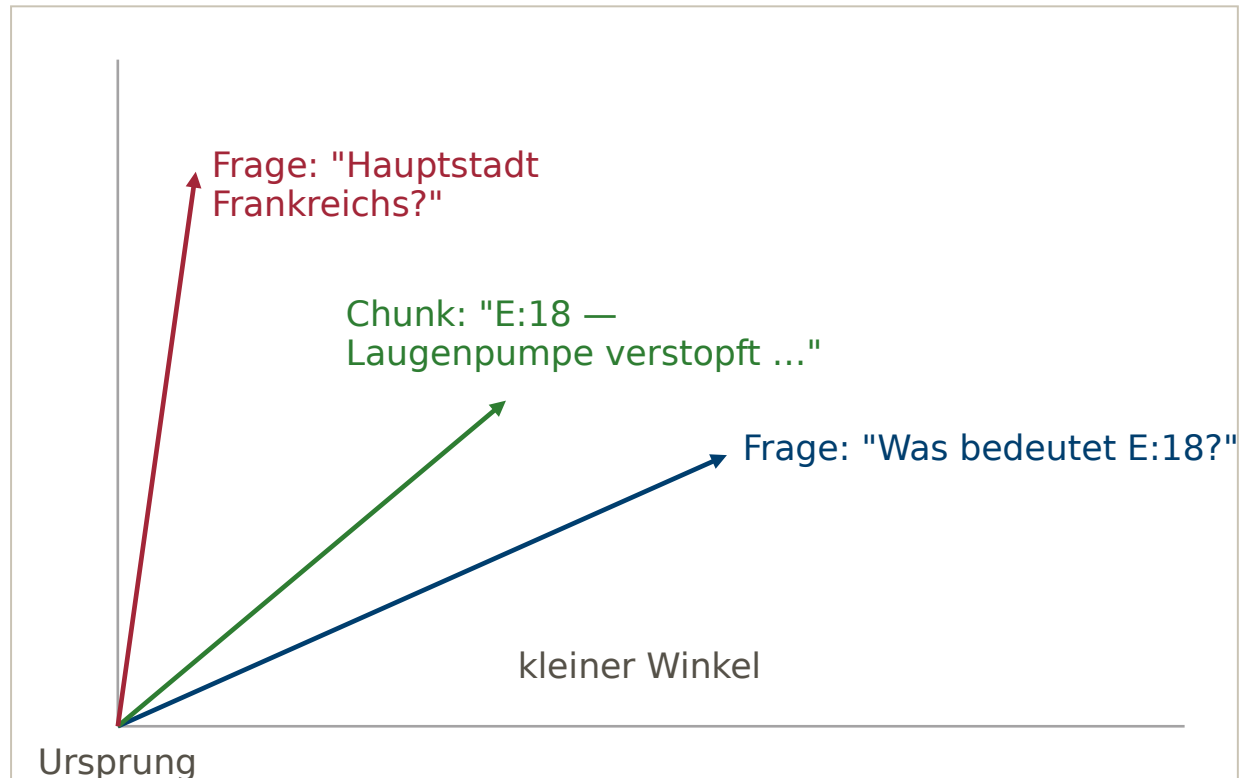
Die Zahlen im Diagramm sind illustrativ, keine gemessenen Werte. Vergleichbar sind die Vektoren nur, weil derselbe Encoder beide erzeugt.

Quelle: huggingface.co/intfloat/multilingual-e5-small (Modellkarte); docs/2026-07-07-optimization-design.md im Repository (Abruf 19.09.2026)

Ähnliche Bedeutung heißt ähnliche Richtung im Vektorraum



Das Embedding-Modell ist so trainiert, dass Texte mit ähnlicher Bedeutung auf Vektoren mit ähnlicher Richtung abgebildet werden — unabhängig von Wortwahl und Sprache. Die 384 Dimensionen lassen sich nicht zeichnen; die Skizze reduziert sie auf zwei und zeigt das Prinzip: Die Frage nach E:18 und die Tabellenzeile zur Laugenpumpe zeigen in dieselbe Richtung, die Frage nach der Hauptstadt Frankreichs woandershin. Gemessen wird der Winkel.



Winkel statt Abstand

Die Kosinusähnlichkeit vergleicht nur die Richtung. Ein langer Abschnitt und eine kurze Frage können deshalb trotzdem nahe beieinanderliegen.

Nähe ist nicht Wahrheit

Nahe Vektoren teilen ein Thema. Ob der Abschnitt die Frage richtig beantwortet, entscheidet erst das Lesen — durch Reranker und Mensch.

Quelle: Eigene Darstellung (schematisch, zwei von 384 Dimensionen); Wang et al. (2024): Multilingual E5 Text Embeddings, arXiv:2402.05672

Kosinusähnlichkeit ordnet Kandidaten, sie belegt keine Antwort



Die Kosinusähnlichkeit misst den Winkel zwischen Fragevektor und Textvektor, unabhängig von deren Länge; LlamaIndex nutzt sie im SimpleVectorStore als Standardmaß. Ein hoher Wert heißt thematische Nähe, nicht, dass der Abschnitt die Frage beantwortet. E:18 und E:23 stehen in derselben Tabelle und liegen sprachlich nahe, bedeuten aber Verschiedenes. Im kleinen Index läuft die Suche als direkter Vergleich mit allen Vektoren, ohne Näherungsverfahren.

$$\cos(q, d) = (q \cdot d) / (\|q\| \cdot \|d\|)$$

1 gleich · 0 rechter Winkel · -1 entgegengesetzt

Hoher Suchscore

Der Text kann zur Frage passen. Seine Aussage muss trotzdem gelesen und geprüft werden — der Score bewertet Nähe, nicht Richtigkeit.

Fehlercodes

E:18 und E:23 liegen im Vektorraum nahe beieinander, haben aber verschiedene Bedeutungen. Deshalb ergänzt ein exakter Lookup die Vektorsuche.

Ähnlichkeitswerte sind keine Wahrscheinlichkeit für eine richtige Antwort.

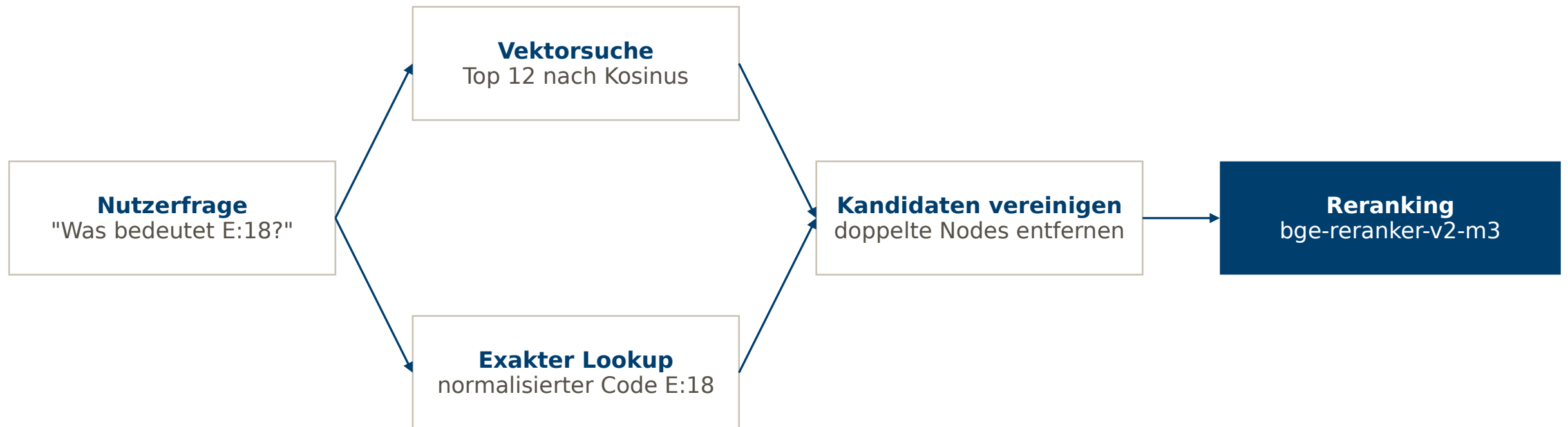
Quelle: llama_index/core/base/embeddings/base.py (SimilarityMode.DEFAULT = Kosinus); huggingface.co/intfloat/multilingual-e5-small (Abruf 19.09.2026)

Suchen, sortieren, ablehnen

Hybrid heißt hier Vektorsuche plus exakter Fehlercode-Lookup



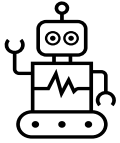
Die Vektorsuche liefert die zwölf ähnlichsten Chunks. Parallel sucht ein exakter Lookup nach normalisierten Fehlercodes — E18, E:18 und "Fehlercode:18" gelten als derselbe Code, E:180 nicht als E:18. Beide Kandidatenmengen werden vereinigt, doppelte Nodes entfernt und an den Reranker übergeben. BM25, Reciprocal Rank Fusion und eine Volltextdatenbank sind nicht implementiert; Hybrid meint in diesem Projekt nur diese Vereinigung.



Ein unbekannter Code darf keine fremde Codebedeutung übernehmen: E:180 ist kein Treffer für E:18.

Quelle: Eigene Darstellung nach rag_engine.py im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Bi-Encoder vergleicht zwei Vektoren, Cross-Encoder liest das Paar



Ein Bi-Encoder wie E5 berechnet für Frage und Text getrennte Vektoren; der Vergleich ist danach nur noch eine Kosinusrechnung, deshalb lässt sich damit ein ganzer Index in Millisekunden durchsuchen. Ein Cross-Encoder wie bge-reranker-v2-m3 bekommt Frage und Text gemeinsam als eine Eingabe und kann jedes Wort der Frage auf jedes Wort des Textes beziehen. Das ist genauer, aber je Paar ein voller Modelldurchlauf — deshalb nur für die Kandidaten.

1 Bi-Encoder — E5, 384 Dimensionen

Frage → Vektor, Text → Vektor, $\cos(q, d)$

Textvektoren werden einmal berechnet und gespeichert. Zur Anfragezeit entsteht nur der Fragevektor; der Vergleich mit allen Chunks ist reine Arithmetik.

Schnell, skaliert auf den ganzen Index; sieht Frage und Text aber nie zusammen.

2 Cross-Encoder — bge-reranker-v2-m3

[Frage; Text] → Modell → Score

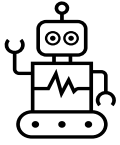
Frage und Text bilden eine Eingabe; die Attention bezieht jedes Wort auf jedes andere. Ergebnis ist ein Logit, mit Sigmoid auf 0 bis 1 abgebildet.

Genauer; aber je Kandidat ein Modelldurchlauf — hier zwölf je Frage.

Arbeitsteilung: Der Bi-Encoder findet Kandidaten, der Cross-Encoder ordnet sie. Beide Modelle laufen lokal.

Quelle: huggingface.co/BAAI/bge-reranker-v2-m3 ("uses question and document as input and directly output similarity"); Modellkarte E5

Der Cross-Encoder liest Frage und Kandidat gemeinsam



Der Cross-Encoder bge-reranker-v2-m3 bewertet Frage und Kandidat als Paar und liefert einen Relevanzscore. Das kostet je Kandidat einen eigenen Modelldurchlauf — deshalb läuft er nur über die zwölf Kandidaten, nicht über den ganzen Index. Der Code bildet die Logits explizit mit Sigmoid auf 0 bis 1 ab und behält die besten fünf Abschnitte. Der Reranker sortiert nur; was die Suche nicht gefunden hat, kann er nicht nachholen.



Mehr Genauigkeit, mehr Rechenzeit

Zwölf Paare je Frage statt eines Fragevektors. Der Reranker braucht spürbar RAM und Rechenzeit; `ENABLE_RERANK=0` schaltet ihn ab.

Sortieren, nicht finden

Der Reranker ordnet nur die Kandidaten. Was Vektorsuche und Lookup nicht geliefert haben, kann er nicht nachträglich finden.

Quelle: huggingface.co/BAAI/bge-reranker-v2-m3 (Modellkarte); `rag_engine.py` im github.com/swrobut/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Sigmoid macht aus einem Logit einen Score zwischen 0 und 1



Der Reranker gibt zunächst ein Logit aus — eine unbegrenzte Zahl, negativ für unpassend, positiv für passend. Die Sigmoid-Funktion $\sigma(x) = 1 / (1 + e^{(-x)})$ bildet sie auf das offene Intervall 0 bis 1 ab: 0 wird zu 0,5, große negative Werte gehen gegen 0, große positive gegen 1. Erst auf dieser Skala ist ein fester Schwellwert wie 0,15 sinnvoll. Der Wert liest sich wie eine Wahrscheinlichkeit, ist aber nicht kalibriert.

$$\sigma(x) = 1 / (1 + e^{(-x)}) \quad \text{Logit } x \text{ beliebig} \rightarrow \text{Score in } (0, 1)$$

Logit x	Score $\sigma(x)$	Lesart
-5	0,007	klar unpassend — abgelehnt
-2	0,119	unter 0,15 — abgelehnt
0	0,500	unentschieden
2	0,881	passend
5	0,993	klar passend

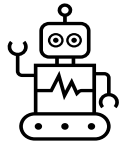
Warum die Schwelle trotzdem Handarbeit ist

Gemessen wurde: Handbuchfragen erreichen Scores ab etwa 0,44, Fremdfragen liegen nahe 0. Die Schwelle 0,15 trennt beides mit Abstand.

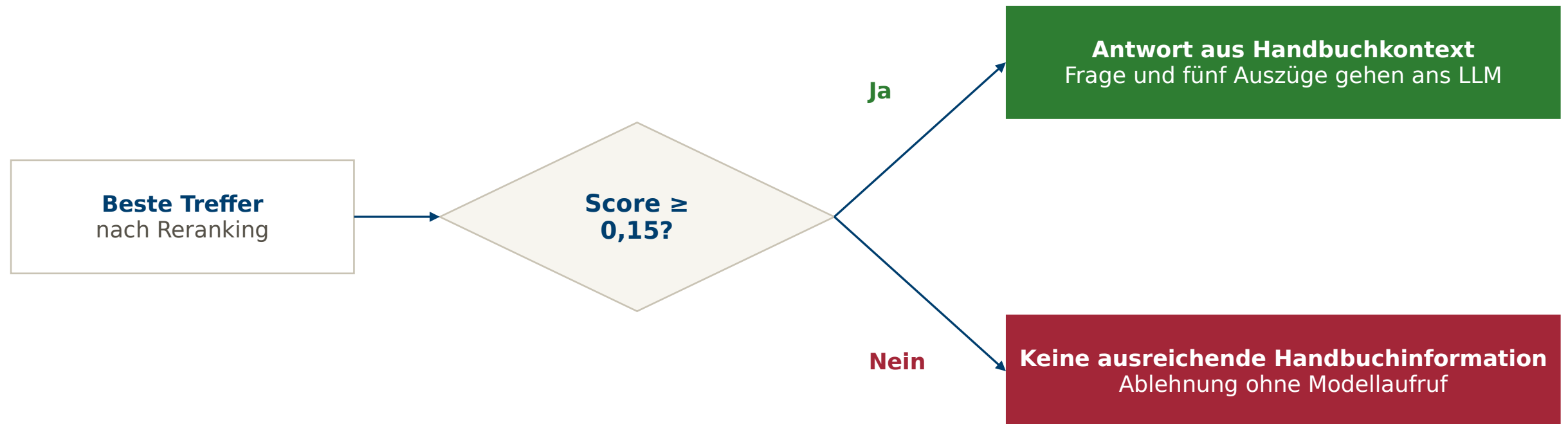
Nicht gegeben: dass 0,88 "88 % richtig" bedeutet. Die Skala ist geordnet, nicht kalibriert.

Quelle: Modellkarte bge-reranker-v2-m3 (normalize=True wendet Sigmoid an); rag_engine.py; Logistische Funktion, Standardmathematik

Unter dem Schwellwert 0,15 antwortet das Modell gar nicht erst



Bevor das LLM aufgerufen wird, prüft die App den höchsten Reranker-Score der Treffer. Liegt er unter `GUARDRAIL_MIN_SCORE = 0,15`, meldet die App "keine ausreichende Handbuchinformation" ohne Modellaufruf. Der Wert ist ein Projektparameter, keine kalibrierte Wahrscheinlichkeit; gesetzt wurde er mit passenden Fragen und echten Gegenbeispielen. Ohne Reranker akzeptiert der Fallback nur exakte Fehlercode-Treffer.



Ein gefundener Abschnitt beweist noch keine belegte Antwort — der Schwellwert filtert nur, was gar nicht zum Handbuch gehört.

Quelle: Eigene Darstellung nach `rag_engine.py` und `server.py` im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Seite 33 ist die Fundseite, 30, 31 und 36 sind Querverweise



Die Fehlercodetabelle steht auf Seite 33 des Handbuchs. E:18 verweist von dort auf die Reinigungsanweisungen der Seiten 30 und 31, E:23 auf den Kundendienst auf Seite 36. Die frühere Quellenfunktion gab diese Verweise als Fundseiten aus. Seit der Korrektur nennt die App nur die Seite aus den Metadaten des Chunks als Fundstelle; alte Verweise sind als Querverweise gekennzeichnet.

Störungen, was tun? de

Hinweise im Anzeigefeld

Anzeige	Ursache/Abhilfe
-P- und	Das Einfüllfenster ist entriegelt. Nachlegen von Wäsche möglich.
-P-	■ Temperatur zu hoch. Warten, bis die Temperatur absinkt. ■ Wasserstand zu hoch. Nachlegen von Wäsche nicht möglich. Ggf. Einfüllfenster sofort schließen. Taste wählen zum Fortsetzen des Programms.
blinkt	■ Evtl. Wäsche eingeklemmt. Bitte Einfüllfenster nochmals öffnen und schließen und Taste wählen. ■ Ggf. Einfüllfenster zudrücken bzw. Wäsche entnehmen und erneut zudrücken. ■ Ggf. Gerät aus- und wieder einschalten; Programm einstellen und individuelle Einstellungen vornehmen; Programm starten.
leuchtet	■ Wasserhahn vollständig öffnen. ■ Zulaufschlauch geknickt/eingeklemmt. ■ Wasserdruck zu gering. Sieb reinigen. → Seite 32
E:18	■ Laugenpumpe verstopft. Laugenpumpe reinigen. → Seite 30 ■ Ablaufschlauch/Abflussrohr verstopft. Ablaufschlauch am Siphon reinigen. → Seite 31
E:23	Wasser in der Bodenwanne, Geräte-Undichtigkeit. Wasserhahn schließen. Kundendienst rufen! → Seite 36
	Kindersicherung aktiviert; deaktivieren. → Seite 26
leuchtet	Zu viel Schaum erkannt, zusätzliche Spülgänge zugeschaltet. Dosieren Sie beim nächsten Waschgang mit gleicher Beladung weniger Waschmittel.
Anzeigelampe für Programm "Trommel Reinigen" blinkt * je nach Modell	Programm Trommel Reinigen 90 °C oder ein 60 °C-Programm zur Reinigung und Pflege der Trommel und des Laugenbehälters durchführen. Hinweise ■ Programm ohne Wäsche durchführen. ■ Pulvervollwaschmittel oder ein bleichehaltiges Waschmittel verwenden. Zur Vermeidung von Schaumbildung nur die Hälfte der vom Waschmittelhersteller empfohlenen Waschmittelmengen verwenden. Benutzen Sie keine Woll- oder Feinwaschmittel. ■ Hinweissignal ein-/ausschalten → Seite 28
andere Anzeigen	Gerät ausschalten, 5 Sekunden warten und wieder einschalten. Erscheint die Anzeige erneut, Kundendienst rufen. → Seite 36

1

Fundstelle Seite 33: Diese Seite steht in den Chunk-Metadaten und wird als Quelle angezeigt.

2

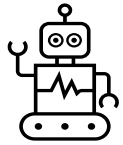
E:18 → Seiten 30 und 31: Reinigungsanweisungen für Laugenpumpe und Ablauf — Querverweis, keine Fundseite.

3

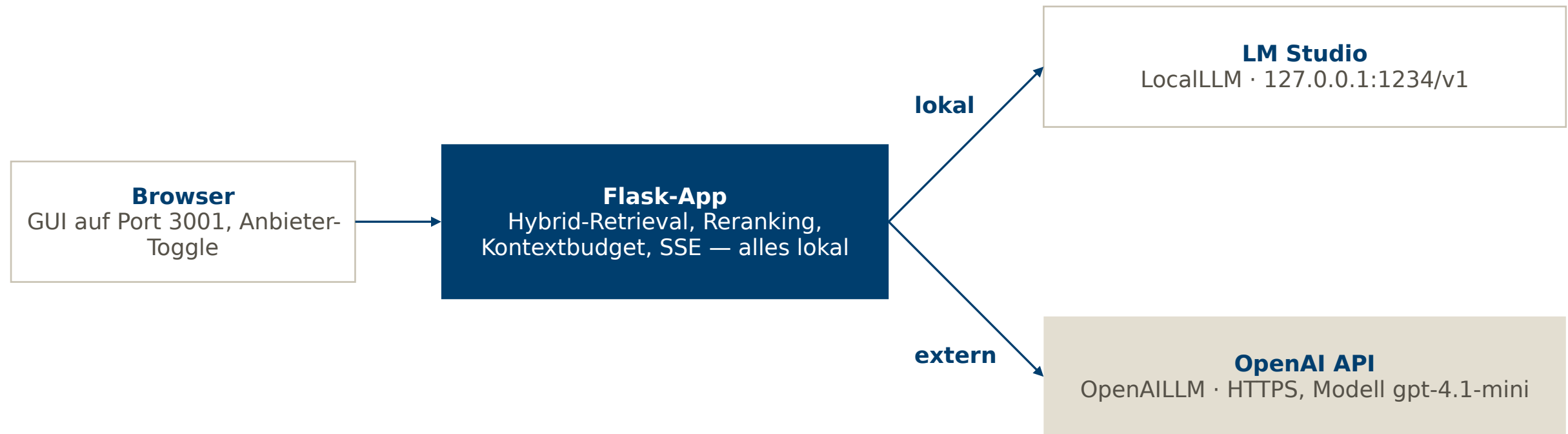
E:23 → Seite 36: Kundendienst — ebenfalls Querverweis. Das Handbuch nennt ihn, der Chunk stammt von Seite 33.

Zwei Antwortmodelle, ein Retrieval-Kern

Ein Server, ein Retrieval-Kern und zwei Modell-Clients



server.py startet eine Flask-App auf Port 3001. Der Umschalter im GUI wählt den Anbieter pro Anfrage: LM Studio über die Klasse LocalLLM oder die OpenAI API über OpenAILLM. Embeddings, Vektorindex und Reranking bleiben in beiden Fällen lokal; an OpenAI gehen nur die Frage und die ausgewählten Handbuchauszüge. Der Anbieter wird mit jeder Anfrage explizit übergeben — ein Wechsel verändert keine Umgebungsvariable und keine Anfrage eines anderen Browsers.



Nach außen gehen nur Frage und ausgewählte Auszüge; der Index verlässt den Rechner nie. OPENAI_MODEL ist konfigurierbar, Voreinstellung gpt-4.1-mini.

Quelle: Eigene Darstellung nach server.py und llm_client.py im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

App und LM Studio gehören auf denselben Rechner



LM Studio stellt einen OpenAI-kompatiblen Server bereit, standardmäßig auf Port 1234, über den die App das geladene Instruct-Modell anspricht. 127.0.0.1 bezeichnet immer den Rechner, auf dem die Python-App läuft: auf dem Mac erreicht sie LM Studio dort, unter Windows zeigt dieselbe Adresse auf Windows. Die Modell-ID wird exakt aus LM Studio übernommen; automatische Auswahl gibt es nur bei genau einem geladenen Chatmodell.



127.0.0.1 ist relativ

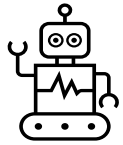
Die Adresse zeigt auf den Rechner der Python-App. Eine App unter Windows erreicht LM Studio auf dem Mac so nicht; dafür braucht es dessen Netzwerk-Endpoint und Serverkonfiguration.

Modell passend zur Hardware

Instruct-Modell und Kontextfenster in LM Studio nach RAM wählen. MLX-Modelle laufen nur auf Apple Silicon und taugen nicht als universelles Default.

Quelle: lmstudio.ai/docs/developer/openai-compat; README.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Nur die Antwort rechnet woanders, Index und Suche bleiben lokal



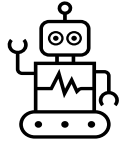
Bei OpenAI rechnet der API-Anbieter die Antwort, bei LM Studio der Modellrechner; Suchindex und Embeddings liegen in beiden Fällen lokal. Übertragen werden Frage und ausgewählte Auszüge — an OpenAI über HTTPS, an LM Studio an den konfigurierten Endpoint. Der Betriebsaufwand verschiebt sich: API-Schlüssel, Netzwerk und Nutzungskosten gegen Modell, RAM oder GPU und Energie. Datenschutz- oder Rechtskonformität folgt aus keinem Betriebsort von selbst.

Eigenschaft	OpenAI API	LM Studio
Antwortberechnung	Beim API-Anbieter	Auf dem Modellrechner
Suchindex und Embeddings	Lokal	Lokal
Übertragene Inhalte	Frage und ausgewählte Auszüge über HTTPS	An den konfigurierten Endpoint
Betriebsaufwand	API-Schlüssel, Netzwerk, Nutzungskosten	Modell, RAM / GPU, Energie
Modellwechsel	OPENAI_MODEL (Standard gpt-4.1-mini)	LOCAL_LLM_MODEL (exakte ID)

Quelle: github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; lmstudio.ai/docs/developer/openai-compatible; developers.openai.com (Abruf 19.09.2026)

PageIndex, Cache und Streaming

PageIndex lässt das LLM Abschnitte auswählen, ohne Vektoren



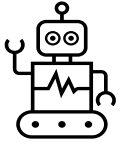
PageIndex ist ein vectorless, reasoning-basierter Ansatz: Statt Vektoren zu vergleichen, liest das gewählte LLM Abschnittstitel und Zusammenfassungen und wählt gültige Node-IDs aus — so, wie ein Mensch ein Inhaltsverzeichnis nutzt. Der gespeicherte Baum enthält 187 Abschnitte mit Titel, Summary, Volltext und ID. Die Fallstudie implementiert eine vereinfachte Auswahl über flache Batches, nicht die agentische Tiefensuche des SDK. Jeder Auswahlaufruf kostet Zeit und bei OpenAI Geld.



Eine Auswahl ist noch keine bestätigte Antwort: Nichtleere ausgewählte Texte sind kein semantischer Relevanznachweis. PageIndex bleibt experimentell.

Quelle: github.com/VectifyAI/PageIndex (README); `pageindex_engine.py` im github.com/swrobut/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Sieben Batches zu 30 Abschnitten, dann eine globale Auswahl



187 Abschnitte geteilt durch 30 je Batch ergeben sieben Auswahlaufrufe. Das LLM antwortet je Batch mit JSON; akzeptiert werden nur IDs, die in diesem Batch vorkamen, ungültiges JSON zählt als leere Auswahl. Bleiben mehr als fünf Kandidaten, folgt ein globaler Aufruf über alle Kandidaten. Vor dieser Korrektur wurde die Liste einfach abgeschnitten, was die zuerst gelesenen Kapitel bevorzugte.

Abschnitte im Baum

187

Abschnitte je Batch

30

Auswahlaufrufe

7

Abschnitte höchstens

5

Batch-Auswahl

Je Batch nur IDs aus diesem Batch; ungültiges JSON zählt als leere Auswahl.

Globale Auswahl

Bei mehr als fünf Kandidaten ein weiterer Aufruf über alle Kandidaten.

Kein Beleg

IDs aus fremden Batches und leere Texte werden verworfen.

Quelle: `pageindex_engine.py` im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; `PAGEINDEX_BATCH = 30`, `PAGEINDEX_MAX_NODES = 5` (Abruf 19.09.2026)

Ein Hash ist ein Fingerabdruck: gleicher Inhalt, gleicher Wert



SHA-256 rechnet aus beliebig langem Inhalt einen Wert von 256 Bit, geschrieben als 64 Hexadezimalzeichen. Derselbe Inhalt ergibt immer denselben Wert; schon ein geändertes Zeichen ergibt einen völlig anderen. Aus dem Wert lässt sich der Inhalt nicht zurückrechnen. Damit lässt sich ohne Vergleich ganzer Dateien prüfen, ob ein Index noch zu seiner Quelle gehört: Stimmt der gespeicherte Hash mit dem frisch berechneten überein, ist die Quelle unverändert.

Eingabe	SHA-256 (erste 16 von 64 Zeichen)	Beobachtung
"E:18"	9099da489dbcb126 ...	eindeutiger Wert für diesen Inhalt
"E:18" (erneut berechnet)	9099da489dbcb126 ...	identisch — Hash ist deterministisch
"E:19"	cf7e5ef11bfa1272 ...	ein Zeichen anders, alles anders

Im Projekt

Der Fingerprint des Vektor-Cache umfasst Markdown, Modellname, Parser-Version, Chunkgröße, Überlappung und Präfixlogik; der PageIndex-Baum trägt den Hash des Markdown.

Was der Hash nicht kann

Er sagt nur "gleich" oder "verschieden", nicht was sich geändert hat. Bei Abweichung wird der Index deshalb komplett neu gebaut.

Quelle: NIST FIPS PUB 180-4, Secure Hash Standard (2015); Beispielwerte mit hashlib.sha256 berechnet, erste 16 von 64 Hexadezimalzeichen

Ein SHA-256-Fingerprint bindet den Index an seine Quelle



Der Vektor-Cache trägt einen SHA-256-Fingerprint über Markdown-Inhalt, Modellname, Parser-Version, Chunkgröße, Überlappung und Präfixlogik; der PageIndex-Baum den Quellhash des Markdown. Beim Laden vergleicht die App den Wert mit den Cache-Metadaten und baut bei Abweichung neu. So kann ein geändertes Handbuch nicht unbemerkt mit einem alten Index laufen. Ein FileLock verhindert, dass mehrere Prozesse gleichzeitig denselben Index aufbauen.



Defekte oder veraltete Caches werden verworfen und neu erzeugt; das Embedding-Modell selbst liegt unter `.cache/embeddings`.

Quelle: Eigene Darstellung nach `rag_engine.py` und `pageindex_engine.py` im github.com/swrobutts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Server-Sent Events sind Klartext: event, data, Leerzeile



Ein SSE-Stream ist eine HTTP-Antwort vom Typ text/event-stream, die offen bleibt. Der Server schreibt Ereignisse als Textblöcke: eine event-Zeile mit dem Typ, eine oder mehrere data-Zeilen mit dem Inhalt, dann eine Leerzeile als Trenner. Der Browser liest die Blöcke, sobald sie ankommen. Die Klasse EventSource kann solche Streams nur per GET öffnen; weil die App die Frage im Body sendet, nutzt sie POST mit fetch und liest den Stream selbst aus.

Beispielstream (verkürzt)

```
HTTP/1.1 200 OK
Content-Type: text/event-stream

event: status
data: {"text": "Suche läuft"}

event: meta
data: {"quelle": "Hinweise im Anzeigefeld · Seite 33"}

event: token
data: {"text": "Die Laugen"}

event: token
data: {"text": "pumpe ist "} ...
```

Warum POST + fetch

EventSource kennt nur GET ohne Body. Frage, Modus und Anbieter stehen aber im JSON-Body. Deshalb fetch(POST) und ein ReadableStream, den index.html selbst in Blöcke zerlegt.

Was ein Abbruch bedeutet

Fehlt am Ende das result-Ereignis oder [DONE], gilt der Stream als abgebrochen. Die App zeigt dann keine halbfertige Antwort als Ergebnis.

Quelle: WHATWG HTML Standard, Server-sent events; MDN, Using server-sent events; server.py (_sse) im Repository

Der Stream zeigt die Quelle, bevor die Antwort fertig ist



Die App sendet den Stream über POST /api/ask_stream. Die Abfolge ist status, meta, token, result und [DONE]: Der Browser sieht den Quellenhinweis, während das Modell noch schreibt. Eine neue Frage oder ein Anbieterwechsel bricht die alte Anfrage ab; eine am Tokenlimit abgeschnittene oder abgebrochene Antwort gilt als Fehler, nicht als Ergebnis. Bereits begonnene Retrieval- oder Anbieterberechnungen können trotzdem noch Kosten verursachen.

status	➔ Suche läuft — Retrieval und Reranking sind gestartet.
meta	➔ Quellenhinweis mit Abschnitt und Fundseite, noch vor dem ersten Antwortwort.
token	➔ Antwortteile, so wie das Modell sie erzeugt; der Browser rendert sie fortlaufend.
result / [DONE]	➔ Vollständiges Ergebnis mit summary, manual_intro und manual_steps; fehlt es, gilt der Stream als abgebrochen.

Quelle: server.py und index.html im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; developers.openai.com, Streaming responses (Abruf 19.09.2026)

Was die Prüfung ergab

Fünf Befunde im RAG-Kern wurden mit Tests behoben



Alle fünf Befunde haben dieselbe Wirkung: Die App hätte Belege angezeigt, die nicht zur Frage gehören — einen falschen Fehlercode, einen abgeschnittenen Abschnitt, eine Tabellenzeile ohne Kopf, eine Seite ohne den zitierten Text, ein früh im Handbuch stehendes Kapitel. Jede Korrektur ist durch einen Regressionstest abgesichert; 70 Tests laufen insgesamt durch. Testabdeckung ersetzt keine fachliche Prüfung der Modellantwort.

Befund	Korrektur
Code-Suche traf Teilzeichenfolgen	Normalisierte, vollständige Fehlercodes; E:18 und E:180 getrennt
Große Texte konnten abgeschnitten werden	Tokenbegrenzte Chunks mit dem E5-Tokenizer, Limitprüfung
Mehrere Tabellen verloren ihre Kopfzeilen	Eigener Header je Tabelle, Fortsetzungszeilen behalten Kontext
Querverweise galten als Fundseiten	Echte Seitenmetadaten; alte Verweise als Querverweise gekennzeichnet
PageIndex bevorzugte frühe Abschnitte	Globale Auswahl über die Batch-Kandidaten statt Abschneiden

Quelle: docs/AUDIT.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM, Prüfbericht vom 19.09.2026

In API und Oberfläche ging es um Angriffsfläche und Ehrlichkeit



Eine Catch-all-Route lieferte beliebige Projektdateien über HTTP aus, Modellantworten landeten ungefiltert in innerHTML, und Eingabe- wie Streamfehler wurden uneinheitlich behandelt. Der Prompt forderte allgemeine Tipps an, obwohl es keine Websuche gab, und fest eingebaute Zahlen im GUI wirkten wie aktuelle Benchmarks. Die Korrekturen begrenzen, was der Server preisgibt und was die Oberfläche behauptet.

Projektdateien über HTTP erreichbar

➔ Nur HTML, Logo und QR-Script sind freigegeben; Regressionstests für private Pfade.

Generierter Inhalt als HTML übernommen

➔ HTML-Zeichen werden vor der Formatierung maskiert; Quelltexte per `textContent`.

Anfrage- und Streamfehler uneinheitlich

➔ JSON-Schema-Prüfung, 2000 Zeichen, 16-KiB-Body, SSE-error-Ereignis, HTTP 400 / 503.

Modellwissen als Internetquelle dargestellt

➔ Der gemeinsame Prompt fordert ausschließlich Handbuchkontext an.

Fest eingebaute Vergleichszahlen

➔ Entfernt; die tatsächlichen Messungen stehen im Prüfbericht.

Quelle: docs/AUDIT.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM, Prüfbericht vom 19.09.2026

Fünf Schichten, fünf Bibliotheksgruppen, eine geprüfte Umgebung



Flask trägt API, Startseite und SSE; LlamaIndex Core mit der HuggingFace-Integration hält Nodes, Vektorsuche und Persistenz; Sentence Transformers und PyTorch rechnen Embeddings und Cross-Encoder; das OpenAI Python SDK spricht die OpenAI API und den kompatiblen LM-Studio-Server an. Docling, LiteLLM, PyMuPDF und pypdf sind optional. requirements.txt fixiert die zusammen getesteten Versionen unter Python 3.12; das Lockfile beschreibt Windows, nicht den Mac.

Schicht	Bibliotheken (geprüfte Version)	Aufgabe
Web	Flask 3.1.3	API, Startseite und SSE
RAG	LlamaIndex Core 0.14.24, HF-Embeddings 0.8.0	Nodes, Vektorsuche und Persistenz
Modelle	Sentence Transformers 6.1.0, PyTorch 2.14.0	Embeddings und Cross-Encoder
LLM-Client	OpenAI Python SDK 2.54.0	OpenAI API und LM-Studio-Kompatibilität
Optional	Docling 2.129.0, LiteLLM 1.101.0, PyMuPDF, pypdf	PDF-Extraktion und PageIndex-Bau

Quelle: requirements.txt, requirements-optional.txt und docs/AUDIT.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Stand 19.09.2026)

Tests prüfen Code und Suche, die Antwortqualität bleibt Handarbeit



Logiktests prüfen Fehlercodes, Cache-Key, Quellen, Tabellenzerlegung und PageIndex-Auswahl ohne Modell. API-Tests prüfen JSON, SSE, ungültige Eingaben, Fehlerfälle und Dateifreigaben. Die Retrieval-Messung stellt zehn Störungsfragen mit Stichworten aus der Anleitung. Die Antwortqualität — Belegtreue, vollständige Warnhinweise, sinnvolle Ablehnung — verlangt echte Modellantworten. Diese fachliche Bewertung ersetzt kein automatisierter Test.

Logiktests

Fehlercodes, Cache-Key, Quellen, Tabellen und PageIndex-Auswahl — ohne Modell.

API-Tests

JSON, SSE, ungültige Eingaben, Fehlerfälle und Dateifreigaben.

Retrieval-Messung

Zehn Störungsfragen mit Stichworten aus der Anleitung, realer Index.

Antwortqualität

Belegtreue, vollständige Warnhinweise und sinnvolle Ablehnung — fachlich, von Hand.

Nur die vierte Ebene beurteilt, was die Studierenden am Ende sehen: die Antwort. Sie fehlt für LM Studio noch ganz.

Quelle: tests/ und eval/ im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; docs/AUDIT.md (19.09.2026)

70 Tests, vier Ablehnungen, eine offene Mac-Verbindung



70 automatisierte Tests laufen durch, darunter Anbieterwechsel, Schlüsselerwaltung und der HTTP-Vertrag zu einem lokalen Testserver. Vier ungeeignete Fragen — zwei Fremdthemen, der unbekannte Code E:180 und eine fremde Geräteart — wurden mit Scores unter 0,03 abgelehnt. Nicht geprüft blieb die reale Antwort aus LM Studio: Der Mac war aus der Windows-Prüfumgebung nicht erreichbar. Die OpenAI-Verbindung wurde nur kurz kontrolliert.

Tests bestanden

70

Gegenbeispiele abgelehnt

4 von 4

Hit@5, Hybrid mit Reranker

100 %

Realer LM-Studio-Lauf

offen

Frage	Höchster Reranker-Score	Ergebnis
Hauptstadt Frankreichs	0,000071	Abgelehnt
Gedicht über den Mond	0,000302	Abgelehnt
Unbekannter Code E:180	0,000267	Abgelehnt
Motor einer Bosch-Spülmaschine	0,027354	Abgelehnt

Quelle: docs/AUDIT.md und docs/evaluation/negative-checks.json im github.com/swrobut/SiemensWashingMachineTroubleShooting_LocalLLM (19.09.2026)

Reranking hebt Hit@1 von 60 auf 90 % bei zehn Fragen



Alle drei Varianten verwenden dieselbe Aufbereitung und denselben E5-Index; die Messung isoliert also Fehlercode-Lookup und Reranking. Der Lookup hebt Hit@5 von 90 auf 100 %, das Reranking Hit@1 von 70 auf 90 % und den MRR von 0,825 auf 0,950. Gemessen wird, ob ein erwartetes Stichwort unter den fünf finalen Treffern steht — ein Such-Proxy für $n = 10$, keine Bewertung generierter Antworten. Laufzeiten enthalten Aufwärmeeffekte und taugen nicht als Benchmark.

Variante (Keyword-Metriken)	Hit@5	Hit@1	MRR	Abdeckung
Vektor allein	90 %	60 %	0,725	73,5 %
Hybrid ohne Reranker	100 %	70 %	0,825	88,5 %
Hybrid mit Reranker	100 %	90 %	0,950	91,0 %

Balkenlänge = Wert, Skala 0 bis 1 in jeder Spalte. Hervorgehoben ist die produktive Einstellung (Hybrid mit Reranker, `ENABLE_RERANK=1`).

Quelle: docs/evaluation/vector.json, hybrid-no-rerank.json, hybrid-rerank.json (19.09.2026); Ablation der Suchkomponenten, $n = 10$

Ein Stichworttreffer ist kein Beleg für eine richtige Antwort



Das Evalset hinterlegt zu jeder Frage erwartete Stichwörter aus den Störungstabellen. Hit@k zählt, ob mindestens eines davon in den Top-k steht, Hit@1 dasselbe für den ersten Treffer; MRR mittelt den Kehrwert des Rangs des ersten Treffers; die Abdeckung misst den Anteil gefundener Stichwörter. Ein einzelnes Wort genügt für einen Treffer. Deshalb sind es Keyword-Proxys — 100 % Trefferquote belegen weder richtige Antworten noch vollständigen Recall.

Keyword-Hit@k

➔ Mindestens ein erwartetes Stichwort in den Top-k Treffern (hier $k = 5$).

Keyword-Hit@1

➔ Mindestens ein erwartetes Stichwort im ersten Treffer.

Keyword-MRR

➔ Mittelwert von $1 / \text{Rang}$ des ersten Stichworttreffers über die zehn Fragen.

Keyword-Abdeckung

➔ Anteil der erwarteten Stichwörter, die in den Ergebnissen vorkommen.

Zehn Fragen, erwartete Stichwörter aus den Tabellen "Hinweise im Anzeigefeld" und "Störungen, was tun?" — Groß- und Kleinschreibung ignoriert.

Quelle: eval/questions.json und eval/run_eval.py im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

MRR 0,950 heißt: neun erste Treffer, einmal Rang 2



Der Mean Reciprocal Rank mittelt über alle Fragen den Kehrwert des Rangs, auf dem der erste passende Treffer steht: Rang 1 zählt 1, Rang 2 zählt 0,5, Rang 5 zählt 0,2, kein Treffer 0. Im Lauf mit Reranker stand bei neun von zehn Fragen der erwartete Abschnitt auf Rang 1; bei der Frage nach auslaufendem Wasser erst auf Rang 2, weil die Warnung vor Wasserschäden vorn lag. $(9 \cdot 1 + 1 \cdot 0,5) / 10 = 0,95$. Die Kennzahl belohnt frühe Treffer, sagt aber nichts über Vollständigkeit.

$$\text{MRR} = 1/|Q| \cdot \sum 1/\text{rank}_i \quad Q = \text{zehn Fragen} \cdot \text{rank}_i = \text{Rang des ersten Stichworttreffers}$$

Wasser läuft aus

Erster Treffer: Rang 2

$$1/2 = 0,5$$

Fehlercode E:18

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Fehlercode E:23

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Kein Abpumpen

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Kein Wassereinlauf

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Tür öffnet nicht

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Kein Programmstart

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Zu viel Schaum

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Kindersicherung

Erster Treffer: Rang 1

$$1/1 = 1,0$$

Trommel reinigen

Erster Treffer: Rang 1

$$1/1 = 1,0$$

$(9 \cdot 1,0 + 1 \cdot 0,5) / 10 = 0,95$. Ohne Reranker lag der Wert bei 0,825, mit Vektorsuche allein bei 0,725 — dort standen die erwarteten Abschnitte häufiger erst auf Rang 2 oder 3.

Quelle: docs/evaluation/hybrid-rerank.json, Feld reciprocal_rank je Frage (19.09.2026); Definition $\text{MRR} = 1/|Q| \cdot \sum 1/\text{rank}_i$

Ein größerer Testbestand braucht Grenzfälle und fachliche Urteile



Zehn Fragen mit Stichwort-Proxys reichen nicht, um Antworttreue zu beurteilen. Ein erweiterter Bestand braucht präzise Codes in allen Schreibweisen, Grenzfälle wie ein falsches Gerätemodell oder fehlende Information, und Fälle mit Sicherheitsbezug, in denen Warn- und Kundendiensthinweise erhalten bleiben müssen. Relevante Abschnitte werden dafür von Hand markiert und Antworten fachlich bewertet. Das sind Empfehlungen, keine Zusage über die heutige Erkennung.

Präzise Codes

E18, E:18, E:180,
unbekannte Codes und
mehrere Codes in einer
Frage.

Grenzfälle

Falsches Gerätemodell,
fehlende Informationen
und fremde Themen.

Sicherheitsbezug

Warnhinweise erhalten,
Kundendiensthinweise
nicht abschwächen.

Messung

Relevante Abschnitte von
Hand markieren,
Antworten fachlich
bewerten.

Warum von Hand: Ein Stichworttreffer erkennt keinen abgeschwächten Warnhinweis. Das kann nur ein Mensch mit dem Handbuch daneben.

Quelle: docs/AUDIT.md, Verbleibende Grenzen (github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM); Gao et al. (2023/2024), arXiv:2312.10997

Betrieb und Ausbau

Ein Datenbankwechsel ist erst mit konkreten Anforderungen begründet



Für eine Anleitung und eine lokale Demo genügt der JSON-Speicher von LlamaIndex. PostgreSQL mit pgvector kombiniert relationale Daten und Vektoren in einer Datenbank, Qdrant bietet Vektorsuche mit Payload-Filtern, Chroma speichert Dokumente, Embeddings und Metadaten. Keines dieser Systeme ist installiert. Den Ausschlag geben Betrieb, Backups, Mehrbenutzerzugriff und Datenmenge; erst dort lohnen Näherungsindizes wie HNSW und IVFFlat.

Speicher	Möglicher Anlass	Status hier
LlamaIndex JSON (SimpleVectorStore)	Einzelnes Dokument und lokale Demo; exakte Suche im Prozess	Im Einsatz
PostgreSQL + pgvector	SQL, Beziehungen und Vektoren gemeinsam; HNSW / IVFFlat	Nicht eingesetzt
Qdrant	Vektorsuche mit Payload-Filtern (must / should)	Nicht eingesetzt
Chroma	Dokumente, Embeddings und Metadaten in einem Store	Nicht eingesetzt

Quelle: github.com/pgvector/pgvector (README); qdrant.tech, Filtering; docs.trychroma.com; developers.llamaindex.ai (Abruf 19.09.2026)

In vier Schritten läuft die App mit LM Studio auf dem Mac



Aus dem Repository-Stamm eine Python-3.12-Umgebung anlegen und requirements.txt installieren. In LM Studio ein zur Hardware passendes Instruct-Modell laden und den lokalen Server starten. profiles/local/.env.example nach profiles/local/.env kopieren und die genaue Modell-ID eintragen; der Endpoint bleibt 127.0.0.1:1234/v1. Dann python server.py starten, http://127.0.0.1:3001 öffnen und "Lokal" wählen. Der erste Suchlauf lädt Modelle und baut den Index.

1 Umgebung

➔ `python3.12 -m venv .venv · source .venv/bin/activate · pip install -r requirements.txt`

2 LM Studio

➔ Instruct-Modell laden, lokalen OpenAI-kompatiblen Server auf Port 1234 starten.

3 Konfiguration

➔ profiles/local/.env mit LOCAL_LLM_ENDPOINT und der exakten LOCAL_LLM_MODEL-ID.

4 Start

➔ `python server.py` · http://127.0.0.1:3001 öffnen · "Lokal" wählen. Die HTML-Datei nicht direkt öffnen.

Weitere Anfragen nutzen den Cache; erst ein geändertes Markdown oder eine geänderte Konfiguration baut den Index neu.

Quelle: README.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; lmstudio.ai/docs/developer/openai-compat (Abruf 19.09.2026)

Der OpenAI-Schlüssel wird im GUI eingegeben, nie in einer Datei



Nach dem Start der App öffnet der Support-Tab "OpenAI-Schlüssel für diese Sitzung". Der Key kommt in ein Passwortfeld; "Für diese Sitzung verwenden" übernimmt ihn, danach ist der Umschalter OpenAI API aktiv. Die Eingabe löst keinen API-Aufruf aus. "Key entfernen", ein Serverneustart oder acht Stunden beenden die Nutzung. profiles/openai/.env enthält nur OPENAI_MODEL; einen OPENAI_API_KEY aus Umgebungsvariablen übernimmt die Web-App nicht.

1 Anwendung starten

2 Schlüssel eingeben

3 OpenAI API wählen

4 Key entfernen

Sitzung statt Datei

Der Key gilt für diese Browsersitzung, höchstens acht Stunden. Das Eingabefeld wird nach der Übernahme geleert.

Kosten entstehen beim Anbieter

Frage und ausgewählte Handbuchauszüge gehen an OpenAI; jede Antwort kostet API-Gebühren. Voreingestelltes Modell: gpt-4.1-mini.

Quelle: README.md und docs/TECHNICAL.md im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM; developers.openai.com, gpt-4.1-mini (Abruf 19.09.2026)

Der Key lebt im Arbeitsspeicher des Servers, acht Stunden lang



POST /api/openai_key legt den Key in SessionKeys ab, einem gesperrten In-Memory-Speicher des lokalen Servers, getrennt je Browsersitzung. Der Browser erhält ihn nie zurück: Das Cookie trägt nur Sitzungs-ID und CSRF-Token, HttpOnly und SameSite=Strict; kein localStorage, kein sessionStorage, keine Datei. Der Server nutzt den Key ausschließlich als Anmeldedatum an der OpenAI API über HTTPS, nie als Promptinhalt. Fehlerprotokolle enthalten nur den Ausnahmetyp.

Passwortfeld im lokalen GUI

Eingabe über POST
/api/openai_key mit CSRF-Token;
das Feld wird danach geleert.

Lokaler Server, Arbeitsspeicher

SessionKeys je Browsersitzung;
Loopback, Host und Origin werden
geprüft. Kein Cookie trägt den Key.

OpenAI API, HTTPS- Anmeldung

Nur als Anmeldedatum; nie im
Prompt, nie in der Antwort, nie im
Protokoll.

Entfernen, Ablauf oder Serverneustart sperren weitere Aufrufe; bereits laufende Anbieteraufrufe können noch zu Ende laufen.

Quelle: session_keys.py und server.py im github.com/swrobut/SiemensWashingMachineTroubleShooting_LocalLLM; flask.palletsprojects.com, Security (Abruf 19.09.2026)

Foliensatz, Prüfbericht und README beschreiben denselben Git-Stand



Eine Änderung am Handbuch oder am Code durchläuft die Tests und die Retrieval-Messung, wird per Git committed und mit GitHub abgeglichen; danach wird der Mac aktualisiert und der lokale Lauf geprüft. Lokale Konfiguration, Modellgewichte und generierte Indizes gehören nicht ins Repository. Ein geändertes Markdown invalidiert Vektorcache und PageIndex-Baum. Die Git-Historie macht die geprüften Änderungen nachvollziehbar.



Nicht im Repository: profiles/*.env, .cache/, storage/ und Modellgewichte. Sie entstehen auf jedem Rechner neu.

Quelle: README.md und .gitignore im github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM (Abruf 19.09.2026)

Belegtreue, PageIndex und Mehrbenutzerbetrieb bleiben offen



Die Quellenanzeige erlaubt die Prüfung, garantiert aber keine durchgehend belegte Modellantwort; dafür fehlt ein größerer, fachlich annotierter Testbestand. Die vereinfachte Batch-Auswahl von PageIndex ist experimentell und kann Text übersehen. Weitere Dokumente brauchen Gerätemodell, Version und Dokument-ID als Filter. Ein produktiver Betrieb braucht Anmeldung, Lastgrenzen und eine fachliche Freigabe — bis dahin lauscht die App nur auf 127.0.0.1.

Belegtreue

Quellenanzeige allein garantiert keine durchgehend belegte Modellantwort.

PageIndex

Die vereinfachte Batch-Auswahl ist experimentell und kann Text übersehen.

Mehr Dokumente

Gerätemodell, Version und Dokument-ID als Filter ergänzen.

Produktiver Betrieb

Authentifizierung, Lastgrenzen und fachliche Freigabe gesondert vorsehen.

Modellantworten ersetzen keine Sicherheits- und Kundendiensthinweise des Originalhandbuchs.

Quelle: docs/AUDIT.md, Verbleibende Grenzen, und docs/TECHNICAL.md, Grenzen (github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM, 19.09.2026)

Paper, Modellkarten, Doku und Repository belegen jede Zahl



Die RAG-Grundlagen stammen aus Lewis et al. (2020) und dem Survey von Gao et al. (2023/2024). Dimensionen, Tokenlimits und Präfixe kommen aus den Modellkarten von multilingual-e5-small und bge-reranker-v2-m3. Speicher, Parser, Streaming und Kompatibilität sind in den Dokumentationen von LlamaIndex, Docling, LM Studio, OpenAI und PageIndex belegt. Projektzahlen — 70 Tests, 187 Nodes, Seite 33, die Messwerte — stammen aus Repository, Prüfbericht und Eval-Protokollen.

RAG-Grundlagen

➔ Lewis et al. (2020), NeurIPS, arXiv:2005.11401 · Gao et al. (2023/2024), arXiv:2312.10997 · Wang et al. (2024), arXiv:2402.05672

Modelle

➔ Modellkarten intfloat/multilingual-e5-small und BAAI/bge-reranker-v2-m3 auf huggingface.co

Frameworks und Standards

➔ LlamaIndex, Docling, LM Studio, OpenAI (gpt-4.1-mini, Streaming), PageIndex · WHATWG SSE · NIST FIPS 180-4 · Flask Security

Projektbelege

➔ github.com/swrobuts/SiemensWashingMachineTroubleShooting_LocalLLM · siemens-handbuch.pdf, Seite 33 · docs/AUDIT.md · docs/evaluation/*.json

Quelle: alle URLs Abruf 19.09.2026; vollständige Adressen in den Sprechernotizen der jeweiligen Folie